

# **Experiment 3: DC-DC Converter for Maximum Power Point Tracking (MPPT) and Charge Control**

## **Part 2: Maximum Power Point Tracking and Charge Control**

ECEN 5517-012

*Khalid Shahba*

*Stewart Rojec*

*BENCH 5*

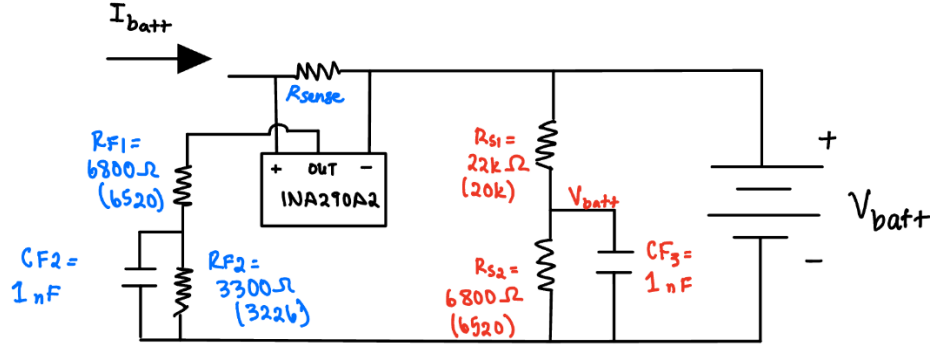
# I. Introduction

The goal of this experiment is to use the buck converter built in the first part of Experiment 3 and make it capable of tracking and operating around the panel's maximum power point via digital control. To accomplish this, we will write an MPPT algorithm to control the duty cycle of the converter, which will be informed by load current and voltage sensing circuitry that we design and build. Then, we will test the converter's power tracking and charge control to validate the design and construction.

## II. Experimental Procedure and Results

### a. Battery Current and Voltage Sensing

#### i. Schematic of Sensing Circuitry



Schematic of Sensing Circuitry (in parentheses are the actual measured values)

#### ii. Reasoning Behind Choices of Component Values

The resistor divider ratios are based on the maximum anticipated values of  $I_{batt}$  and  $V_{batt}$  corresponding to 3.3V once divided down. Their individual values were based on the tradeoff between power dissipation and RC time constant for gates on the ADC. The capacitor value was based on allowing dynamic changes to occur seamlessly without allowing harmonic noise to pollute our measurements.

$V_{batt\_max} = 14V$ , divided down must be 3.3V so  $\frac{R_{S2}}{R_{S2} + R_{S1}} = \frac{3.3}{14}$ . Power consumed must be  $< 10$  mW and  $R_{S1} < 50k\Omega$  for non-delayed ADC response so 6800 and 22000 were chosen.  $C_{F3} = 1nF$  creating a 3dB cutoff frequency of 7.234kHz, well below the fundamental switching harmonic of 100 kHz and all of its higher order harmonics.

$I_{batt\_max} = 10A$ , which going through the INA290A2 current sense amplifier corresponds to 10V out. Divided down, that must be 3.3V so  $\frac{R_{F2}}{R_{F1} + R_{F2}} = \frac{3.3}{10}$ . Power consumed again must be  $< 10$  mW and  $R_{F1} < 50k\Omega$  for ideal ADC response. Thus, 3300 and 6800 were chosen. Again,  $C_{F2} = 1$  nF creating a cutoff frequency of 23.4kHz, low enough to attenuate switching harmonics.

#### iii. Expressions for ADC Input Voltages (using measured R's)

$$V_{IbattADC} = 50R_{sense}I_{Batt} \left( \frac{R_{F2}}{R_{F2} + R_{F1}} \right) = 0.331I_{batt}$$

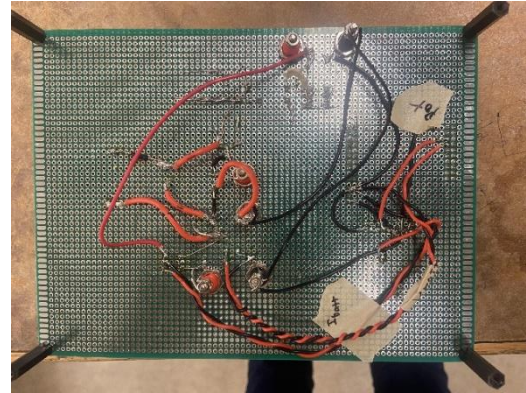
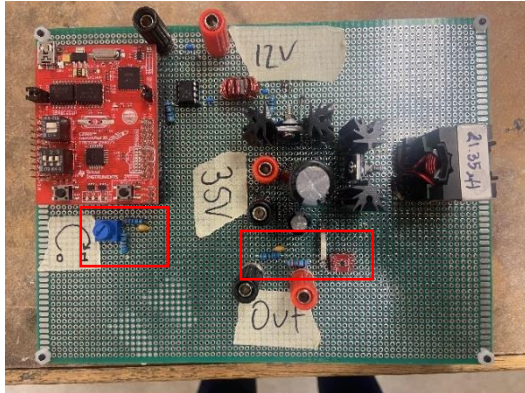
$$V_{VbattADC} = V_{batt} \left( \frac{R_{S2}}{R_{S2} + R_{S1}} \right) = 0.24585V_{batt}$$

The measured ADC values are simply these expressions multiplied by the conversion factor  $\frac{4095}{3.3V}$

## b. Testing of Battery Current and Voltage Sensing

### i. Photograph of Top and Bottom Side of Complete Converter with Sensing Circuit

The constructed sensing circuit and twisted pairs connecting it to the microcontroller's ADC are shown below.



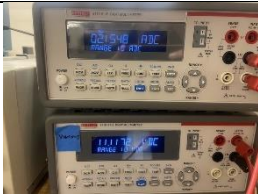
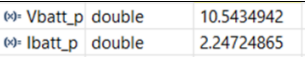
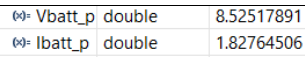
Sensing Circuit Pictures (Front and Back)

### ii. Description of Testing/Debugging/Evaluation Procedure

To test the sensing circuitry, we powered the buck converter with the bench supply and loaded it with the rheostat. After confirming in CCS that the sensing circuitry was reporting data, we loaded a test program onto the microcontroller that switched between two duty cycles of 65% and 80% every 10s, since those duty cycles are sufficiently different to produce a noticeable discrepancy in power behavior. We then compared the load voltage and current reported by the sensing circuitry with the results from the benchtop measurement equipment to determine how accurately the sensing circuit could capture battery current and voltage, and more importantly, their product. We applied changes to the nominal resistor values in the code to account for the discrepancies we saw and were able to reconcile the measured power and code-reported power to within roughly 3% maximum error between them. The user code is in appendix A.

### iii. Test Data and Interpretation of Results

The table below shows the data from benchtop testing of our sensing circuit.

| Duty Cycle | Measured Vbatt and Ibatt                                                                                                   | Measured Pbatt | Debugger-Reported Vbatt and Ibatt                                                                                          | Code Pbatt | Error |
|------------|----------------------------------------------------------------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------------------------------|------------|-------|
| 80%        | <br>Ibatt = 2.1548A<br>Vbatt = 11.1172V | 23.955W        | <br>Ibatt = 2.247A<br>Vbatt = 10.5435V | 23.447W    | 2.1%  |
| 65%        | <br>Ibatt = 1.7467A<br>Vbatt = 9.0132V  | 15.743W        | <br>Ibatt = 1.828A<br>Vbatt = 8.525V   | 15.584W    | 1%    |

We found that our sensing circuit undershoots voltage by about 5.1% to 5.4% and overshoots current by about 4.2 to 4.7%. Although the sensing circuitry and code doesn't provide highly accurate figures in these individual values for voltage and current, the errors in voltage and current seem to offset each other so that the reported power is only about 1-2% lower than the power measured by the product of the values on the benchtop multimeters. Additionally, since the error in each voltage and current measurement seems to be linear across the range of values tested, the inaccuracy in sensed battery voltage and current will not actually lead to malfunction of the MPPT algorithm. This is because systematic under-reading will still find the peak power point. The microcontroller will still arrive at the maximum power, it will just slightly misrepresent what that MPPT voltage and current actually are.

### c. MPPT and Charge-Control Development: Bench Testing

#### i. Description of MPPT Algorithm with Charge-Control Feature and Copy of Commented Code

The MPPT algorithm we decided to implement combines the perturb and observe method with a periodic sweep. The algorithm keeps track of elapsed time by incrementing a counter variable in the ADC ISR, which is called at the same rate as our switching frequency of 100 kHz. Each increment represents roughly 10  $\mu$ s, so the algorithm uses that to track the time elapsed. To begin, we sweep the duty cycle to find the optimum power point every 60 seconds, aka 6 million calls to the ADC ISR. This initial sweep is rough at only 5% per step in duty cycle in order to minimize the amount of time it spends on suboptimal duty cycles. Given the fact that we incorporate transient delays of 195ms to allow for  $I_{batt}$  and  $V_{batt}$  to settle and use 5ms to average hundreds of steady state measurements of  $I_{batt}$  and  $V_{batt}$ , this initial sweep takes 4 seconds. For the remaining 56 seconds before the next rough sweep, we perturb and observe. The algorithm perturbs the duty cycle by increments of only 0.17% now, again with 200 ms of transient delay and averaging. The essence of perturbing and observing is to remember which direction the last perturbation was and whether power increased or decreased. If power increased, the duty cycle goes in the same direction as the previous perturbation and if it decreased, it goes the opposite direction. **Ideally, this creates a 3-level oscillation when plotting duty cycle over time, where the middle level is the true peak duty cycle.** (See II.c.iii, 1 $\Omega$  test to see very well define 3-level oscillation).

Our charge control was implemented via a while loop. This was a challenging thing to implement because we had to probe the battery voltage while it was being charged and under various loads. We could've implemented a delay where we measure the non-charging loaded battery voltage but the transients to measure would've been far too long and taken too much time out of our allotted perturb and observe period. Thus, we probe battery voltage after every perturbation just once with no averaging or delays. If the measured voltage, which itself includes a small error, is greater than 13V, we set the duty cycle to 0 and we enter a while loop that measures the battery voltage every 200 ms again. There is a correction factor included that incorporates the error from the battery voltage when it's charging of roughly 1.3V. **The unloaded battery voltage in our tests corresponded to leaving and entering this period of no power delivery whenever the actual open circuit voltage of the battery was at a threshold of 12.6V.** The actual turn off voltage of the converter would only go up as the battery gets loaded further because 12.6V is the loaded battery voltage in that case and when unloaded, it could be as high as 12.9V or more even. Thus, the threshold is adjustable using our constants assuming different constant loads but was sufficient to prove the safety feature works. Indeed, the best way to know a battery's state of charge is to keep track of how many amp-hours it has supplied, but we lack the necessary hardware to do so.

Here is a snippet of all our relevant added code to realize this algorithm. The only thing not shown is `adc_isr_ctr` incremented in the interrupt service routine function. The setup and other provided functions are also excluded:

```
// User code
pulse_width = 0;           // PWM duty cycle is D = pulse_width/period
D_max = 0;
update_duty(pulse_width);
double RF1 = 6520.0;       //custom measured resistor divider values and other constants
double RF2 = 3226.0;
double RS1 = 20000.0;
double RS2 = 6520.0;
double ADC_factor = 3.3/4095.0;
double Rsense = 0.02;
Ibatt = 0;                 // Ibatt, Vbatt and Pbatt are all initialized to 0 as they are counter variables in our average scheme
Vbatt = 0;
Pbatt = 0;
adc_isr_ctr = 0;           // this is the variable counting how many times we enter the ADC ISR (1 time every 10 us).
avg = 0;
unsigned int ctrl_save_time = 0;
while (adc_isr_ctr < 19500){} // 195 ms delay once we initialize and have D = 0.
while (adc_isr_ctr >= 19500 && adc_isr_ctr < 20000)
{
    Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); //for the final 5 ms of our 200ms delay, we take averages of the measured Ibatt and Vbatt
    Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
    avg++;
}
Ibatt_p = Ibatt/avg; //at the end of summing up hundreds of Ibatts and Vbatts and counting up exactly how much "hundreds" is, we can create
Vbatt_p = Vbatt/avg; //solid data points that refresh every 200ms in the form of Ibatt_p, Vbatt_p and Pbatt_p.
Pbatt_p = Ibatt_p*Vbatt_p;

Pbatt_max = Pbatt_p; //right away, set this first measured value as the max to beat.
unsigned int n = 0; //this variable is how we can perturb and observe while still keeping track of adc_isr_ctr reaching 60s.
unsigned int c = 0; //this variable keeps track of the direction we went in the previous perturbation.
adc_isr_ctr = 70000000; // we set the adc_isr_ctr manually high to force a sweep when the code first loads.
while (1)
{
    // do a general sweep of duty cycle every minute from 0 to 90 (pulse_width of 0 to 540) as a starting point
    if (adc_isr_ctr >= 60000000)
    {
        n = 0;
        Pbatt_max = 0;
        for(pulse_width = 1; pulse_width < 570; pulse_width += 30){
            update_duty(pulse_width);
            adc_isr_ctr = 0;
            Ibatt = 0;
            Vbatt = 0;
            while(adc_isr_ctr < 19500){}
            avg = 0;
            while(adc_isr_ctr >= 19500 && adc_isr_ctr < 20000){
                Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); // see commented lines 125-142 for the explanation of
                Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2; // this averaging technique
                avg++;
            }
            Ibatt_p = Ibatt/avg;
            Vbatt_p = Vbatt/avg;
            Pbatt_p = Ibatt_p*Vbatt_p;
            if(Pbatt_p > Pbatt_max){
                Pbatt_max = Pbatt_p; // at the end of each step in the sweep, reassign a
                D_max = pulse_width; // MPPT Duty Cycle (D_max) every time a new peak battery
                                   // power is reached
            }
            if(pulse_width > 510){
                adc_isr_ctr = 0; // at the end of the sweep set ADC_isr_ctr back to 0.
            }
        }
        // now we have the peak D that gets cycled every minute and we perturb and observe
        // until 60 seconds reached
        c = 0; //initially start at a decrease from the D_max arrived at in rough sweep.
        pulse_width = D_max-3;
        update_duty(pulse_width);
    }
    if(D_max >= 571) // this is a safety feature that says if somehow, D_max has become greater than 95%, restart the sweep
    {
        // by resetting adc_isr_ctr to 60000000.
        adc_isr_ctr = 60000000;
        D_max = 0;
        pulse_width = 0;
    }
    n++; //now that the first perturbation has occurred, increment n.
    Ibatt = 0;
    Vbatt = 0;
    avg = 0;
    adc_isr_ctr = (n-1)*20000.0 - 500.0; //manually reset adc_isr_ctr to allow an even transient for all perturbations and observations
    while(adc_isr_ctr < n*20000.0 - 500.0){}
    while(adc_isr_ctr >= n*20000.0 - 500.0 && adc_isr_ctr < n*20000.0){
        Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0)); // again, use the final 5ms to average hundreds of measurements
        Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
        avg ++;
    }
    Ibatt_p = Ibatt/avg;
    Vbatt_p = Vbatt/avg;
    Pbatt_p = Ibatt_p*Vbatt_p;
    if(Pbatt_p > Pbatt_max && c == 0) // if we decreased DC and went up in power, do it again. And reset Pbatt_max as always.
    {
        D_max = pulse_width - 1;
        pulse_width = D_max;
        c = 0;
        Pbatt_max = Pbatt_p;
        update_duty(pulse_width);
    }
}
```

```

}
else if(Pbatt_p < Pbatt_max && c == 0) // if we decreased DC and went down in power, go up in duty cycle.
{
    D_max = pulse_width + 1;
    pulse_width = D_max;
    c = 1;
    Pbatt_max = Pbatt_p;
    update_duty(pulse_width);
}
else if(Pbatt_p > Pbatt_max && c == 1) // if we increased DC and went up in power, do it again.
{
    D_max = pulse_width + 1;
    pulse_width = D_max;
    c = 1;
    Pbatt_max = Pbatt_p;
    update_duty(pulse_width);
}
else if(Pbatt_p < Pbatt_max && c == 1) // if we increased DC and went down in power, go down in duty cycle.
{
    D_max = pulse_width - 1;
    pulse_width = D_max;
    c = 0;
    Pbatt_max = Pbatt_p;
    update_duty(pulse_width);
}
else // if by some rarity, the measured power is identical to the previous one, change nothing and maintain
{
    Pbatt_max = Pbatt_p;
}
}
while(Vbatt_p > 13){ // here is our charge control check at the end of every perturbation and observation.
    pulse_width = 0;
    update_duty(pulse_width);
    ctrl_save_time = adc_isr_ctr; //after turning off duty cycle, wait for 200ms before determining if staying is correct
    while(adc_isr_ctr - ctrl_save_time < 200000){
        Vbatt_p = sensed_voltage*ADC_factor*(RS2+RS1)/RS2 + 1.3; //add a correction factor that incorporates the loss of charging current added voltage
        pulse_width = 0;
        update_duty(pulse_width);
    }
}
}
}

```

## ii. Description of Testing, Debugging, and Evaluation Procedure

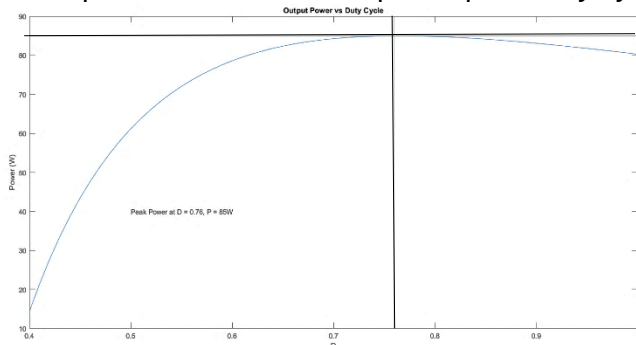
To test the maximum power algorithm at the bench before using it with the panel, we need to create an approximation of the panel with a resistor  $R_{in}$  and a voltage source  $V_{in}$  that allows the converter to track a maximum power point. To calculate these values, we used the maximum power operating point of 85 W and applied the following equation from the Thevenin equivalent:

$$85 = \frac{V_g(V_{in} - V_g)}{R_{in}}$$

Through a first order derivative it can be shown that  $V_g = \frac{V_{in}}{2}$  will yield maximum power. Thus, at 85 W,  $V_{in} = 34 \text{ V}$  &  $V_g = 17 \text{ V}$ . Then, by plugging the peak expression for  $V_g$  back in, we see the following:

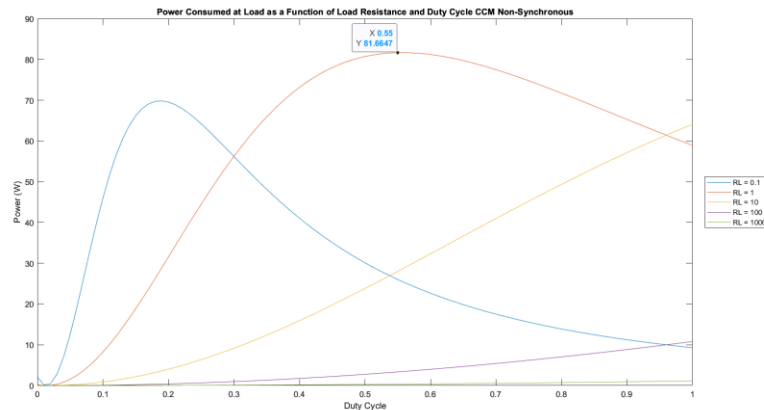
$$85W = \frac{34^2}{4R_{in}}, \text{ so } R_{in} = 3.4 \Omega$$

The first test we implemented was with the battery as the load. Assuming the lossless duty cycle expression  $V_g = \frac{V_o}{D}$  and an approximation of  $V_o$  as 13V when charging, the following plot predicts the maximum power point duty cycle:



**Roughly speaking, a 76% Duty cycle should be expected in our benchtop battery-as-load tests assuming a constant output voltage of 13V and a MPP at 17V.**

Since battery voltage is unstable and hard to predict, we implemented an additional test with the Thevenin equivalent. This test used a much more stable output load resistance to find the Thevenin peak operating duty cycle. A buck converter can only achieve MPPT for load resistances equal to or less than the source's own Thevenin resistance. See the script in appendix B for how we derived the following plot. After incorporating losses, **the script predicts that for a  $1\ \Omega$  load, the maximum power deliver occurs at a duty cycle of 55% for a Thevenin resistance of  $3.4\ \Omega$  and 34V input voltage:**



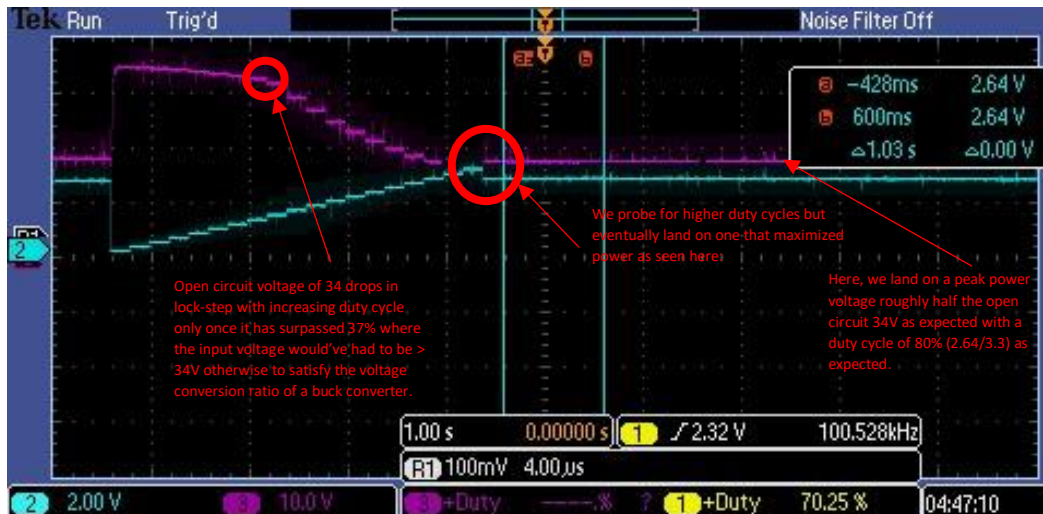
- iii. Capture of Oscilloscope + Microcontroller Waveforms of Converter Input Voltage, LPF PWM Signal, Max Duty Cycle, and Max Power at Peak Power

### ***Battery Connected as Load:***



Close up of Peak Power Point Tracking During Perturb and Observe Step: Scope Captures,  $V_g = 17.7\text{V}$ ,  $V_{pwm} = 2.72\text{V}$ .

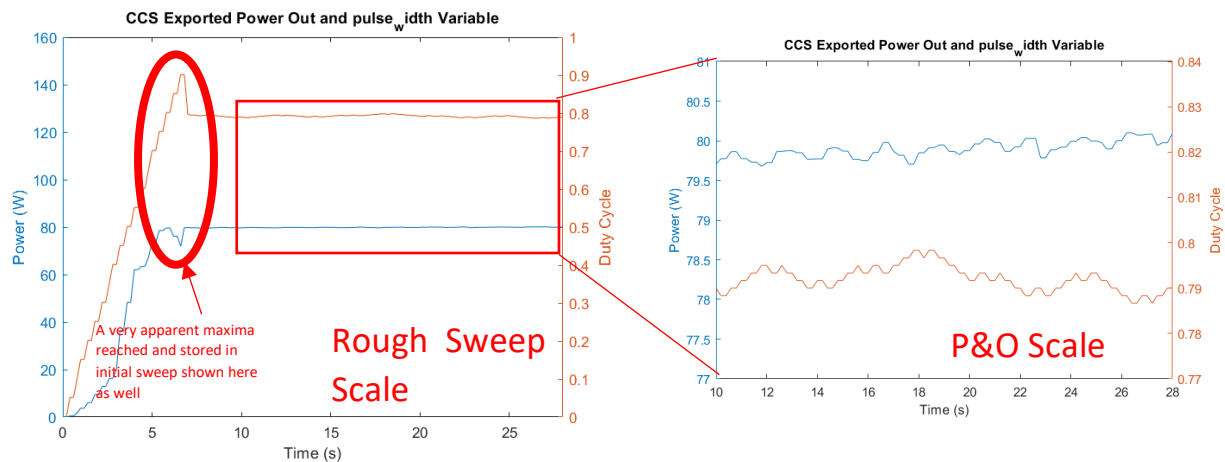
Time:  $10\ \mu\text{s}/\text{div}$ , **Purple**:  $V_g$  ( $V_{pv}$ )  $5\text{V}/\text{div}$ , **Yellow**: J6.1 Duty Cycle  $2\text{V}/\text{div}$ , **Cyan**: Filtered  $V_{pwm}$   $2\text{V}/\text{div}$



Large-Time scale Showing  $V_g$  (Vpv) changing during a periodic sweep and locking onto a peak power duty cycle. (Scale too small to show perturb and observe)

Time: 1 s/div, **Purple:**  $V_g$  (Vpv) 10V/div, **Cyan:**  $V_{pwm}$  2V/div

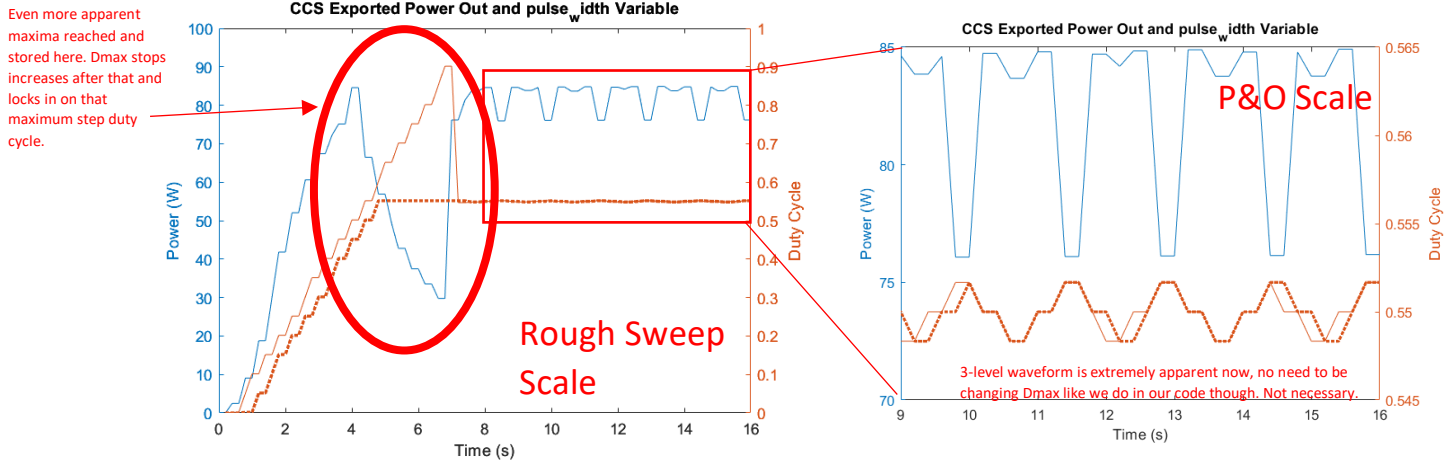
These trends can also be microscopically observed when using MATLAB and exported data that the CCS debugger logs for us.



Zoomed in Extracted Data from CCS Environment showing P&O and Initial Rough Step During "Battery as Load" test

**Blue:** Power to the Load, **Orange:** Overlaid pulse\_width variable

### 1 $\Omega$ Connected as Load:



Zoomed in Extracted Data from CCS Environment showing much better-defined P&O and Initial Sweep behavior

Blue: Power to the 1  $\Omega$  Load, Orange: Overlaid pulse\_width variable (Solid), Dmax variable (Dotted)

#### iv. Explanation of MPPT Data in II.c.iii

The 1  $\Omega$  test shows it even more vividly than the battery-as-load test but both showcase a working rough sweep stage and a working perturb and observe stage. We predicted an MPPT duty cycle of 76% for 13V out. The battery, while charging, measured with a voltage closer to 13.6 (having to pick up voltage through ammeter). Such an output voltage actually makes our prediction 80% for a peak power duty cycle. **Indeed, we oscillate between 79 and 80%.** Likewise, we predicted 55% duty cycle to be the peak power point when 1  $\Omega$  was our load. **Our three-level, quite perfect P&O scale plot from this test shows that the middle level, the actual MPPT point, occurs exactly at 55%.** The swings in power for such small changes in duty cycle for the 1  $\Omega$  test are quite shocking though, going from 85W to 75W simply by being the smallest possible subdivision of duty cycle over the peak operating point.

#### v. Input Power, Output Power, and Converter Efficiency Under MPPT Conditions

##### Battery-as-Load MPPT Benchtop Tests

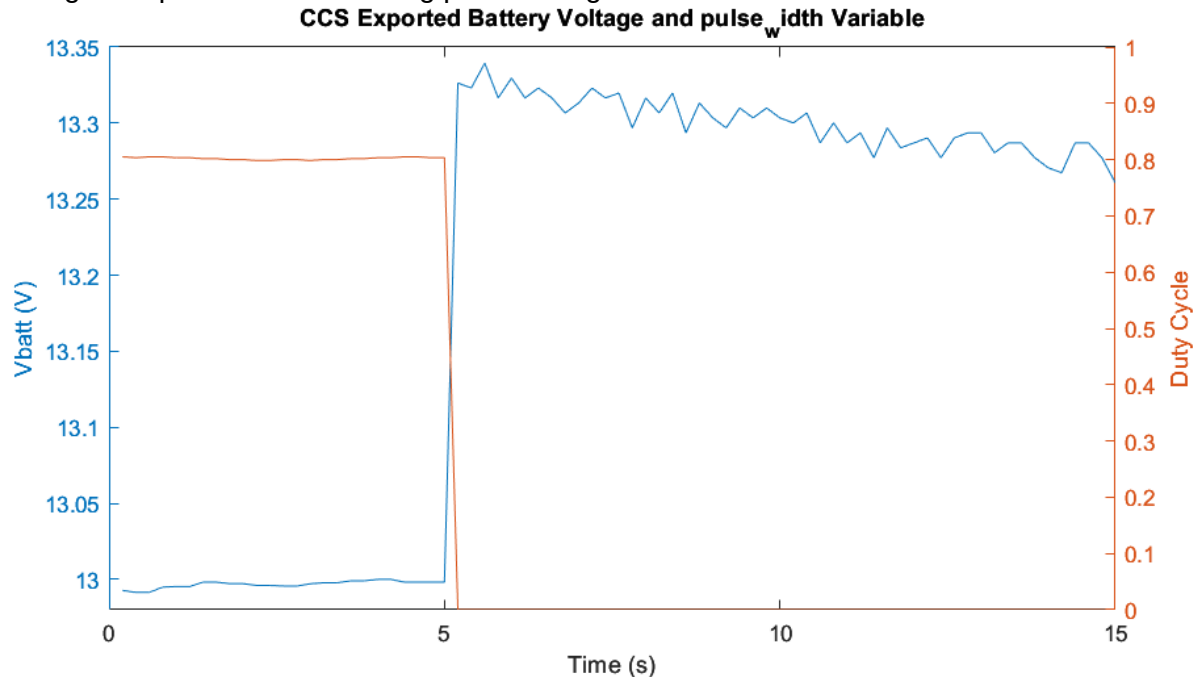
| Input Voltage | Input Current | Input Power | Output Voltage | Output Current | Output Power | Efficiency |
|---------------|---------------|-------------|----------------|----------------|--------------|------------|
| 17.2V         | 4.89A         | 84.108W     | 13.18V         | 6.08A          | 80.1344W     | 95.28%     |

##### 1 $\Omega$ Load MPPT Benchtop Test

| Input Voltage | Input Current | Input Power | Output Voltage | Output Current | Output Power | Efficiency |
|---------------|---------------|-------------|----------------|----------------|--------------|------------|
| 17.6V         | 4.87A         | 85.712W     | 8.8V           | 8.63A          | 75.944W      | 88.6%      |

## vi. Charge Control Validation

Again, the CCS environment was extremely helpful in proving our charge control protection was working as expected. The following plots were generated to show it worked:



This figure shows the battery voltage during charging reaching the overvoltage threshold at 5 second mark in **Blue**. Upon doing so, the battery voltages spikes thanks to the correction implemented to ensure that charging does not commence again until the equivalent value for open circuit voltage is reached. For the final 10 seconds, the battery voltage is slowly decreasing either due to simply approaching steady state, or because it is powering on board auxiliary converters and the kill-a-watt, etc. And of course, in this state of high charge, the duty cycle, shown in **orange**, is set to 0 from its perturb and observe state at 5 seconds until the battery voltage + correction falls under 13V.

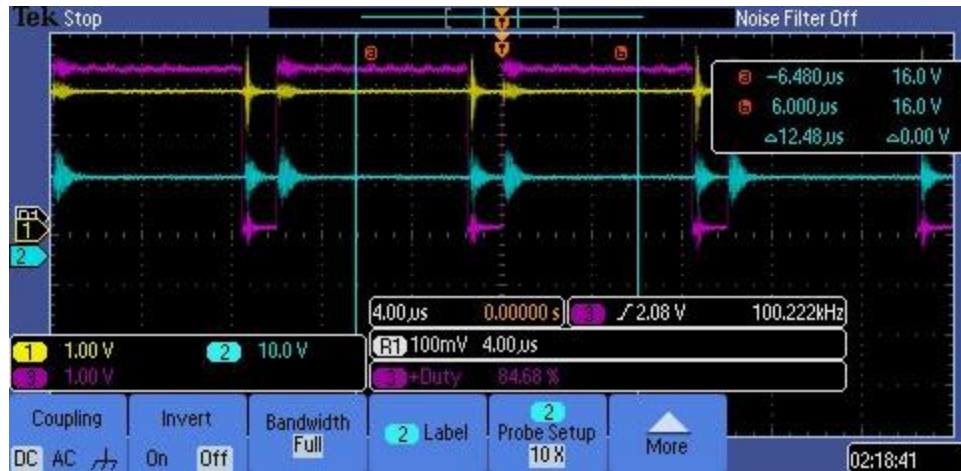
When we load the battery again, the voltage drops beneath the 13V threshold. If it has been less than 60 seconds since it locked out, it will begin perturbing and observing starting from a duty cycle of 0 which takes a very long time and will hit the 60 second large-step sweep cycle before actually reaching peak power. If it has been more than 60 seconds, which is usually the case as the battery needs time to discharge below the lockout threshold, the algorithm will start normally with a rough sweep followed by the much more fine perturb and observe.

## d. MPPT and Charge-Control Development: Complete System Test (Outdoors Testing)

### i. Time of Day and Solar Irradiation Data

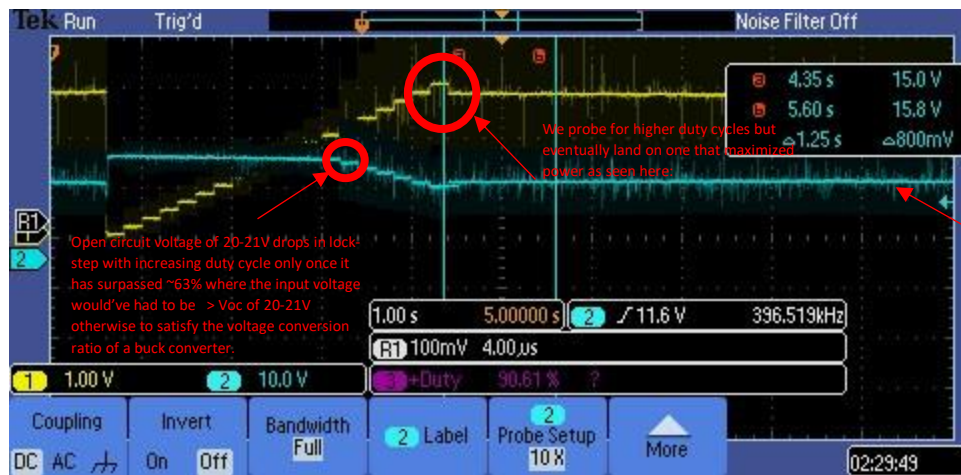
| Time of Day | Measured Maximum Solar Irradiance |
|-------------|-----------------------------------|
| 3:15 PM     | 1086 W/m <sup>2</sup>             |

### ii. PV Voltage and LPF Vpwm Signals



Close up of Peak Power Point Tracking During Perturb and Observe Step: Scope Captures,  $V_{pv} = 16.0V$ ,  $V_{pwm} = 2.8V$ .

Time: 4  $\mu s$ /div, **Cyan:** ( $V_{pv}$ ) 10V/div, **Purple:** J6.1 Duty Cycle 1V/div, **Yellow:** Filtered  $V_{pwm}$  1V/div



Large-Time scale Showing  $V_{pv}$  changing during a periodic sweep and locking onto a peak power duty cycle. (Scale too small to show perturb and observe)

Time: 1 s/div, : **Cyan:**  $V_g$  ( $V_{pv}$ ) 10V/div, **Yellow:**  $V_{pwm}$  1V/div

One thing worth mentioning is that our MPPT script uses the smallest possible perturbation in duty cycle possible. The sweep, however, checks duty cycles at a much larger increment of 5%. If we landed on a sweep step that was still far away from the maximum power point, the perturb and observe often took longer than expected to reach the actual peak power point. This is fixed simply by increasing the step size in our perturb and observe, but since this didn't happen too often, we left the perturbation at the minimum size to preserve precision in MPPT.

### iii. Converter Input Power, Power Delivered to Battery, and Efficiency with no Shading

We took multiple measurements of power delivery data over multiple days to see how consistently our MPPT performed. Here is the data. All peak power points occurred at around 84-86 % Duty Cycle during the P&O stage.

| Date/Time       | SI (W/m <sup>2</sup> ) | PV Voltage (V) | PV Current (A) | PV Power (W) | Difference between Exp2 MPP | Battery Voltage (V) | Battery Current (A) | Battery Power (W) | Efficiency |
|-----------------|------------------------|----------------|----------------|--------------|-----------------------------|---------------------|---------------------|-------------------|------------|
| 2/28<br>2:55 PM | 1080                   | 15.3           | 4.66           | 71.298       | -7.333                      | 12.27               | 5.2                 | 63.804            | 89.49%     |
| 2/28<br>2:58 PM | 1080                   | 15.88          | 4.34           | 68.92        | -9.712                      | 12.26               | 5.03                | 61.67             | 89.48%     |
| 2/28<br>3:01 PM | 1077                   | 16.0           | 4.34           | 69.44        | -9.191                      | 12.26               | 5.03                | 61.67             | 88.81%     |
| 3/01<br>3:47 PM | 1060                   | 16.4           | 4.1            | 67.24        | -11.39                      | 12.41               | 4.92                | 61.06             | 90.80%     |
| 3/01<br>3:55 PM | 1055                   | 16.3           | 4.03           | 65.69        | -12.94                      | 12.4                | 4.86                | 60.26             | 91.7%      |

### iv. Comparison of II.d.iii to DET

We redid our DET as our Experiment 2 DET data occurred under different loading conditions for our battery. When loaded identically, the DET data at 3:25 PM on 2/28 was 12.18V and 4.77A, so 58.1W. Thus, we were able to extract, using our algorithm at least 7.6W more and even as much as 13.2W more out of our panel to deliver to the battery.

### v. Results with 4 Cells Shaded vs DET, Discussion

When shaded, the following power delivery data was collected:

| PV Voltage | PV Current | PV Power | Battery Voltage | Battery Current | Battery Power | Efficiency |
|------------|------------|----------|-----------------|-----------------|---------------|------------|
| 13.87V     | 0.22A      | 3.0514W  | 11.99V          | 0.25A           | 2.975W        | 97.5%      |

In this stage, our converter has entered DCM. If this were a synchronous buck, it would have become a boost converter in the opposite direction, powering the panel. However, the Schottky diode prevents power from flowing out of the battery. Instead, the PV Voltage is forced into being 13.87V but can't supply much power because in creating such a voltage while shaded, current had to be driven through reverse-biased PN Junctions in the shaded part of the panel (and not go through the bypass diode of the 18 cells in series the shaded cells are a part of) leading to little available power. In DCM, the conversion ratio of output voltage and input voltage is different and that means that the plot of duty cycle vs output power does not reach a peak point within the 0% to 90% duty cycle range we have allowed the converter to operate between. This results in a duty cycle of 90% in these tests as we never actually reach a peak in our MPPT algorithm because there is no peak. A buck boost converter would be able to manifest a lower voltage, higher current input model that could supply more power when compared in particular to DET.

### III. Appendix

#### a. II.b.ii Sensing Circuitry Validation Code

The only thing not shown is adc\_isr\_ctr incremented in the interrupt service routine function. The setups and other provided functions are also excluded:

```
// User code
pulse_width = 480;          // PWM duty cycle is D = pulse_width/period
D_max = 0;
update_duty(pulse_width);
double RF1 = 6520.0;
double RF2 = 3226.0;
double RS1 = 20000.0;
double RS2 = 6520.0;
double ADC_factor = 3.3/4095.0;
double Rsense = 0.02;
Ibatt = 0;
Vbatt = 0;
Pbatt = 0;
adc_isr_ctr = 0;
avg = 0;
while (adc_isr_ctr < 999500){} // 10s delay
while (adc_isr_ctr >= 999500 && adc_isr_ctr < 1000000)
{
    Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0));
    Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
    avg++;
}
Ibatt_p = Ibatt/avg;
Vbatt_p = Vbatt/avg;
Pbatt_p = Ibatt_p*Vbatt_p;
//Ibatt_max = Ibatt;
//Pbatt_max = Pbatt;
//unsigned int n = 0;
unsigned int c = -1; //0 means we went down in DC, 1 means we went up

while (1)
{
    if(c == -1)
    {
        avg = 0;
        Ibatt = 0;
        Vbatt = 0;
        pulse_width = pulse_width - 90;
        c = 1;
        while (adc_isr_ctr < 999500){} // 10s delay
        while (adc_isr_ctr >= 999500 && adc_isr_ctr < 1000000)
        {
            Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0));
            Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
            avg++;
        }
        adc_isr_ctr = 0;
        Ibatt_p = Ibatt/avg;
        Vbatt_p = Vbatt/avg;
        Pbatt_p = Ibatt_p*Vbatt_p;
    }
    else if(c == 1)
    {
        avg = 0;
        pulse_width = pulse_width + 90;
        c = -1;
        Ibatt = 0;
        Vbatt = 0;
        while (adc_isr_ctr < 999500){} // 10s delay
        while (adc_isr_ctr >= 999500 && adc_isr_ctr < 1000000)
        {
            Ibatt += sensed_current*ADC_factor*(RF2+RF1)/RF2 * (1/(Rsense*50.0));
            Vbatt += sensed_voltage*ADC_factor*(RS2+RS1)/RS2;
            avg++;
        }
        adc_isr_ctr = 0;
        Ibatt_p = Ibatt/avg;
        Vbatt_p = Vbatt/avg;
        Pbatt_p = Ibatt_p*Vbatt_p;
    }
}
```

## b.II.c.ii Secondary Thevenin MPPT Test Script Predictor for $1\Omega$ as load

```

clear all
clc
syms Ig IC1 IC2 IL CD VD Vo Vg RL Rs Rsense Ron RD D L fs Ron1 Ron2 Rideal D1 D2 D3 Rth

Ts = 1/fs

Dp = 1-D

% CCM Steady State Non-Dynamic EQNS (based on SSA and IVSB and CCSB
IC1_eqn1 = Ig - IL
VL_eqn1 = Vg - Ig*(Rth) - IL*Ron - IL*Rs - Vo
IC2_eqn1 = IL-Vo/RL

IC1_eqn2 = Ig
VL_eqn2 = 0 - IL*Rs - VD - IL*RD - Vo
IC2_eqn2 = IL - Vo/RL

eqn2 = 0 == D*IC1_eqn1 + Dp*IC1_eqn2
Ig_s = simplify(expand(solve(eqn2, Ig)))

eqn1 = 0 == D*VL_eqn1 + Dp*VL_eqn2
Vo_s = simplify(expand(solve(eqn1, Vo)))
Vo_s = simplify(expand(subs(Vo_s, Ig, Ig_s)))
Vo_s_dummy = subs(Vo_s, [VD RD Rsense Rs Rth Ron], [0 0 0 0 0])
I_RL_dummy = Vo_s_dummy/RL

eqn3 = 0 == D*IC2_eqn1 + Dp*IC2_eqn2
IL_s = simplify(expand(solve(eqn3, IL)))
% RL MUST be non-zero and must not be infinity either
IL_s_dummy = subs(IL_s, [VD RD Rsense Rs Rth Ron Vo], [0 0 0 0 0 Vo_s_dummy])
IL_ss = simplify(expand(subs(IL_s, Vo, Vo_s)))
% rewriting inductor current so that it isnt a function of inductor current
IL_ss_s = simplify(expand(solve((IL_ss - IL) == 0, IL)))
Vo_ss_s = simplify(expand(subs(Vo_s, IL, IL_ss_s)))
IRL_ss_s = simplify(expand(Vo_ss_s/RL))

simplify(expand(subs(Vo_ss_s, [VD RD Rs Rsense Ron], [0 0 0 0 0])))
simplify(expand(subs(IL_ss_s, [VD RD Rs Rsense Ron], [0 0 0 0 0])))
simplify(expand(subs(IRL_ss_s, [VD RD Rs Rsense Ron], [0 0 0 0 0])))

% Writing out generic expression for the power out from CCM with a diode,
% efficiency, and input resistance. The efficiency and power out assume a
% constant Vg even though that isn't the case. However, the input
% resistance is a function of vg and vg is a function of input resistance.
% The value of Vg and input resistance will be known through testing and
% the value of D will be established therein. Still worth it to show the
% graphs for a fixed Vg.
Pout_CCM_DIODE = simplify(expand(Vo_ss_s * IRL_ss_s))
Efficiency_CCM_Diode = simplify(expand(Pout_CCM_DIODE)/(Vg*IL_ss_s))
Input_Res_Diode = simplify(expand(Vg/IL_ss_s))

% now here are graphs to show nonsynchronous vs synchronous CCM boost
% converter stuff
D_arr = 0:0.01:1

Vg_ex = 34
VD_ex = 0.47
RD_ex = 0
Rs_ex = 0.002
Rsense_ex = 0.02
Ron_ex = 0.028
Rth_ex = 3.4

% Vout graph for the variables set above
Vout_CCM_Diode_G = subs(Vo_ss_s, [Vg RD Rs Rsense Ron VD Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex Rth_ex])
for RL_Z = logspace(-1, 3, 5)
    Vout_CCM_Diode_G = subs(Vo_ss_s, [Vg RD Rs Rsense Ron VD RL Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex RL_Z Rth_ex]);
    str = "RL = " + RL_Z;
    plot(D_arr, double(subs(Vout_CCM_Diode_G, D, D_arr))/Vg_ex, 'DisplayName', str)
    hold on
end
xlabel('Duty Cycle')
ylabel('Conversion Ratio')
title('Conversion Ratio as Function of Load Resistance and Duty Cycle CCM Non-Synchronous')
legend('location','eastoutside')
hold off

%pout graph for the variables set above
Pout_CCM_Diode_G = subs(Pout_CCM_DIODE, [Vg RD Rs Rsense Ron VD Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex Rth_ex])

```

```

for RL_Z = logspace(-1, 3, 5)
    Pout_CCM_Diode_G = subs(Pout_CCM_DIODE, [Vg RD Rs Rsense Ron VD RL Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex RL_Z
Rth_ex]);
    str = "RL = " + RL_Z;
    plot(D_arr, double(subs(Pout_CCM_Diode_G, D, D_arr)), 'DisplayName',str)
    hold on
end
xlabel('Duty Cycle')
ylabel('Power (W)')
title('Power Consumed at Load as a Function of Load Resistance and Duty Cycle CCM Non-Synchronous')
legend('location','eastoutside')
hold off

% efficiency for the variables set above
Efficiency_CCM_Diode_G = subs(Efficiency_CCM_Diode, [Vg RD Rs Rsense Ron VD Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex
Rth_ex])
for RL_Z = logspace(-1, 3, 5)
    Efficiency_CCM_Diode_G = subs(Efficiency_CCM_Diode, [Vg RD Rs Rsense Ron VD RL Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex
VD_ex RL_Z Rth_ex]);
    str = "RL = " + RL_Z;
    plot(D_arr, double(subs(Efficiency_CCM_Diode_G, D, D_arr)), 'DisplayName',str)
    hold on
end
xlabel('Duty Cycle')
ylabel('Efficiency')
title('Efficiency as a Function of Load Resistance and Duty Cycle CCM Non-Synchronous')
legend('location','eastoutside')
hold off

% input resistance for the variables set above
Input_Res_CCM_Diode_G = subs(Input_Res_Diode, [Vg RD Rs Rsense Ron VD Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex Rth_ex])
for RL_Z = logspace(-1, 3, 5)
    Input_Res_CCM_Diode_G = subs(Input_Res_Diode, [Vg RD Rs Rsense Ron VD RL Rth], [Vg_ex, RD_ex Rs_ex Rsense_ex Ron_ex VD_ex
RL_Z Rth_ex]);
    str = "RL = " + RL_Z;
    plot(D_arr, double(subs(Input_Res_CCM_Diode_G, D, D_arr)), 'DisplayName',str)
    hold on
end
xlabel('Duty Cycle')
ylabel('Input Resistance')
title('Input Resistance as a Function of Load Resistance and Duty Cycle CCM Non-Synchronous')
legend('location','eastoutside')
hold off

```